

Phase 11: Safety Guardrails

Hospital Patient Helpdesk Chatbot

Python module: 06_rag_pipeline/11_safety_guardrails.py **Jupyter notebook:**

13_notebooks/11_safety_guardrails.ipynb **Purpose:** Block or replace unsafe medical advice, diagnosis, dosage recommendations, emergency-handling errors, prompt injection, unsupported citations, and recognized sensitive-data leakage.

1. Phase objective

Phase 11 is the final deterministic safety boundary between the Phase 10 RAG chain and patient-facing applications. Earlier phases attempt to route and prevent unsafe behavior. This phase independently checks the final question and answer, applies approved replacement messages, removes unsupported provenance, redacts recognized sensitive values, and records every decision.

This layer is defense in depth. It is not a medical device, diagnostic system, emergency service, or substitute for clinical review.

2. Folder placement

```
hospital_patient_helpdesk_chatbot/
|-- 01_data/
|   |-- processed/
|   |   |-- 10_rag_answers.json
|   |   |-- 11_guarded_answers.json
|   |   |-- 11_guardrail_report.json
|   |   |-- 11_guardrail_audit.csv
|   |   |-- 11_safety_test_results.json
|   |   |-- 11_failed_guardrail_checks.json
|   |   |-- plots/
|   |       |-- 11_guardrail_actions.png
|   |       |-- 11_rule_trigger_counts.png
|-- 06_rag_pipeline/
|   |-- 10_rag_chain.py
|   |-- 11_safety_guardrails.py
|   |-- 11_safety_guardrails.md
|   |-- 11_safety_guardrails.pdf
|-- 12_tests/
|   |-- test_guardrails.py
|-- 13_notebooks/
|   |-- 11_safety_guardrails.ipynb
|-- README.md
```

All generated Phase 11 files and plots use the 11_ prefix.

3. Input file

01_data/processed/10_rag_answers.json

The module validates and consumes the Phase 10 final-answer contract:

Field	Safety use
answer_id	Stable audit and failure identifier.
question	Checked for emergency, unsafe medical, injection, and sensitive-data patterns.
answer	Checked for dosage, diagnosis, unsupported citation, and sensitive-data patterns.
mode	Preserved for safe answers or replaced by a safer mode.
citations	Verified against the answer's own source list.

Field	Safety use
sources	Defines the only permitted citation labels.
retrieval_confidence	Propagated as retrieval metadata, not medical certainty.
safety_labels	Preserved as upstream routing signals.
provider and model	Preserved for traceability.

4. Output files

Numbered artifact	Description
11_guarded_answers.json	Patient-facing answer contract after final policy enforcement.
11_guardrail_report.json	Action, risk, triggered-rule, test, configuration, and output summaries.
11_guardrail_audit.csv	One decision row per Phase 10 answer without storing a second copy of answer text.
11_safety_test_results.json	Expected and actual results for the built-in adversarial cases.
11_failed_guardrail_checks.json	Answer IDs and sanitized processing errors.
11_guardrail_actions.png	Counts of pass, override, block, and redact decisions on real Phase 10 answers.
11_rule_trigger_counts.png	Rule triggers produced by the adversarial test suite.

The plots demonstrate operational behavior and policy coverage. They do not prove clinical safety.

5. Guarded answer schema

```
{
  "answer_id": "ANSWER-009",
  "question": "I have severe chest pain. What is wrong with me?",
  "answer": "Contact local emergency services immediately ...",
  "mode": "emergency",
  "citations": [],
  "sources": [],
  "retrieval_confidence": "medium",
  "safety_labels": ["emergency", "unsafe_medical_advice"],
  "provider": "offline",
  "model": "offline-grounded-v1",
  "safety_flag": true,
  "guardrail_action": "override",
  "risk_level": "critical",
  "triggered_rules": ["GR-001_EMERGENCY_ROUTING"],
  "guardrail_reasons": ["The question contains a possible emergency signal."],
  "guardrail_version": "1.0"
}
```

When a guardrail replaces an answer, citations and sources are cleared because the approved replacement is policy text rather than a statement derived from retrieved evidence.

6. Actions and risk levels

Actions

Action	Meaning
pass	The answer remains unchanged.
override	Unsafe content is replaced with an approved emergency or medical-refusal response.
block	Security-seeking or unverified content is replaced with a safe refusal.
redact	Recognized sensitive values are replaced with typed placeholders.

Risk levels

Risk	Typical use
low	Safe grounded answer passed all checks.
medium	Sensitive data was detected and redacted.
high	Diagnosis, dosage, injection, or grounding violation.
critical	Possible emergency requiring immediate routing.

Risk labels represent guardrail policy severity. They are not medical triage scores.

7. Rules and execution order

Rules are ordered so the most urgent handling wins.

GR-001_EMERGENCY_ROUTING

Detects selected emergency phrases such as severe chest pain, inability to breathe, stroke signs, heavy bleeding, overdose, unconsciousness, or self-harm language. It immediately replaces the answer with instructions to contact local emergency services or go to the nearest emergency department. It does not attempt diagnosis.

GR-002_PROMPT_INJECTION

Blocks requests to ignore prior instructions, reveal system prompts, disclose credentials, expose developer messages, or jailbreak the assistant.

GR-003_UNSAFE_MEDICAL_REQUEST

Intercepts questions requesting diagnosis or dosage advice and replaces the response with an approved clinician/pharmacist referral.

GR-004_DOSAGE_OUTPUT

Detects generated instructions containing a numeric medication amount and an administration instruction or frequency. The answer is replaced with the approved medical refusal.

GR-005_DIAGNOSIS_OUTPUT

Detects direct diagnostic claims such as “you have” or “your diagnosis is.” The answer is replaced rather than edited because partial diagnostic content could remain unsafe.

GR-006_GROUNDING_FAILURE

Blocks grounded answers that have no citation or contain a citation not present in their own source list. The result is changed to an insufficient-context response and all unverified provenance is removed.

GR-007_SENSITIVE_DATA

Redacts recognized API keys, Social Security numbers, email addresses, and medical-record-number patterns using typed placeholders. This is a limited pattern-based control, not a complete de-identification system.

8. Python module code sections

Policy constants and patterns

Approved replacement messages, action values, risk ordering, rule patterns, and citation syntax are centralized near the top of the module so policy behavior is visible and reviewable.

Configuration and contracts

GuardrailConfig controls citation enforcement, sensitive-data redaction, and prompt-injection blocking. SafetyDecision, GuardedAnswer, and GuardrailRunResult define auditable typed boundaries.

Input validation

load_rag_answers() checks file existence, JSON-list structure, required Phase 10 fields, and source/citation list types before processing begins.

Pattern helpers and redaction

matches_any() applies compiled patterns. redact_sensitive_text() replaces recognized values without retaining them in failure output. max_risk() preserves the highest risk when multiple non-terminal checks fire.

Ordered policy evaluation

evaluate_answer() applies emergency, injection, unsafe-request, unsafe-output, grounding, and redaction rules in order. Terminal high-risk question rules return immediately with approved replacement text.

Final transformation and validation

apply_guardrails() builds the downstream guarded-answer contract. validate_guarded_answer() verifies non-empty output, citation integrity, emergency wording, action/rule consistency, and safety-flag consistency.

Adversarial suite

adversarial_cases() defines nine synthetic cases. run_adversarial_tests() records expected action, actual action, expected rule, triggered rules, and pass/fail status. The suite covers all seven rule IDs plus safe pass-through.

Batch artifacts and plots

run_guardrail_evaluation() checks all Phase 10 answers independently, writes protected answers and audits, runs the adversarial suite, creates plots, and writes the aggregate report.

Command-line execution

```
python 06_rag_pipeline/11_safety_guardrails.py
```

Custom input and output locations can be supplied with --answers and --output-dir.

9. Notebook code sections

The notebook:

1. resolves the project from the workspace, project, or notebook directory;
2. imports the shared Python module;
3. loads and validates the 12 Phase 10 answers;
4. demonstrates unchanged safe pass-through;
5. displays emergency, diagnosis, and dosage overrides;
6. runs and asserts all nine adversarial cases;
7. executes the complete Phase 11 batch;
8. validates action, risk, failure, and test totals; and
9. displays both numbered plots.

10. Notebook versus Python module

Topic	11_safety_guardrails.ipynb	11_safety_guardrails.py
Main purpose	Policy walkthrough, demonstrations, assertions, and plots.	Reusable final-safety implementation for API and batch workflows.
Rules	Imports the module's rules.	Owns ordered rules, patterns, messages, and decisions.
Inputs	Uses the Phase 10 sample output.	Accepts any compatible Phase 10 answer file.
Outputs	Displays selected results and reports inline.	Writes protected JSON, audit CSV, tests, failures, report, and plots.
Testing	Runs all adversarial cases interactively.	Exposes testable functions and the adversarial suite.
Automation	Run cells in sequence.	CLI or imported <code>apply_guardrails()</code> call.

The notebook contains no duplicate policy implementation, so it remains aligned with the Python module.

11. Automated tests

`12_tests/test_guardrails.py` contains pytest coverage for:

- safe grounded pass-through;
- emergency routing;
- dosage and diagnosis requests;
- prompt-injection blocking;
- unsupported-citation blocking; and
- sensitive-data redaction.

The built-in adversarial suite additionally tests generated dosage and generated diagnosis claims.

12. Validation results

The sample Phase 10 run produced:

- 12 input answers;
- 12 guarded answers;
- 0 failed checks;
- 9 unchanged passes;
- 3 medical-safety overrides;
- 1 critical emergency decision;
- 2 high-risk unsafe-medical-request decisions; and
- 9 of 9 adversarial tests passing.

The three overridden records contain approved policy messages and no citations or source claims.

13. Limitations and deployment requirements

- Pattern rules cannot recognize every language, typo, euphemism, medical phrase, or contextual risk.
- Sensitive-data patterns are not complete de-identification and can produce false positives or false negatives.
- An emergency phrase list is not a clinical triage instrument.
- Guardrails do not establish that retrieved hospital information is current or correct.
- New languages, providers, departments, and use cases require new reviewed tests.

- Production systems need clinician-approved policy, privacy and security review, monitoring, incident response, human escalation, access control, retention limits, and recurring evaluation.
- Patients must be told that the chatbot is informational and not emergency care or clinical decision support.

When there is any possibility of an emergency, the system should direct the patient to local emergency services rather than attempting to assess or explain the condition.